POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
Faculty of Information Technology

INTELLIGENT SYSTEM DEVELOPMENT · LABORATORY EXPERIMENT 02

A Learning Seller against a Rule-Following Seller

Q-learning and hand-written rules compete in a simulated two-shop e-commerce market

Student	**Bùi Nguyên Hoàng Việt**
Student ID	**B23DCDT285**
Class	E23CNPM02
Semester	I, 2025 – 2026
Submission	Individual project
Lecturer	Assoc. Prof. Dinh Que Tran, Ph.D.
Contact	quetd@ptit.edu.vn

Hanoi, 2026

Contents

1 Overview **2**

2 The Market and the Two Sellers **2**

2.1 Structure of the simulation . . . 2

2.2 What the model already tells us . . . 3

2.3 Seller A: tabular Q-learning . . . 4

2.4 Seller B: rule-based . . . 4

3 Method **4**

3.1 Program layout . . . 4

3.2 Settings . . . 5

3.3 Evaluation protocol . . . 5

4 Findings **5**

4.1 Experiment A – one baseline run (seed 42) . . . 6

4.2 Experiment B – the learning curve . . . 7

4.3 Experiment C – learning rate . . . 8

4.4 Experiment D – exploration . . . 9

4.5 Experiment E – paired comparison over ten seeds . . . 10

5 Answers to the Ten Laboratory Questions **10**

Q1 Modelling the agents and the environment . . . 10

Q2 Learning-based versus rule-based decisions . . . 11

Q3 The Q-learning update . . . 11

Q4 Exploration versus exploitation . . . 11

Q5 Reward design and business objectives . . . 12

Q6 Measuring performance . . . 12

Q7 Learning rate and stability . . . 13

Q8 Interaction between sellers and non-stationarity . . . 13

Q9 Limitations of the simulation and a modified market . . . 14

Q10 Designing a better agent . . . 14

6 Reflection **16**

7 Conclusion **16**

Appendix: Reproducing the Results **16**

References **17**

1 Overview

Two online shops sell the same product. At every time step each shop chooses between keeping its price and giving a 20% discount. A discount brings in more buyers, but every sale earns less and the other shop loses customers. I built this market as a small simulation and placed two very different decision makers in it:

- a **Q-learning seller** (reinforcement learning) that begins with an empty value table and learns from the profit it observes;
- a **rule-based seller** that applies three hand-written *if–then* rules and never adapts.

The assignment asks for the concepts of agent, environment, state, action, reward and policy to be explained, for both sellers to be implemented in a two-agent simulation, for them to be compared with common metrics, and for the strengths and limits of each approach to be discussed.

Contribution beyond the reference program. The reference code trains the learner against the rule once and reports one number. I kept its market, its Q-learning update and its rule exactly as given, and added what is needed to make conclusions trustworthy (Section 3): a *paired* evaluation in which both kinds of seller occupy the same seat, face the same rival and live through the same demand sequence; ten independent seeds for every configuration; fixed-ε and fully greedy exploration settings; and three additional studies that answer Questions 8–10 with measurements.

> **Headline results.**
>
> - Against the rule-based rival the best possible policy is to *never discount* (expected profit 35 per step), and the rule-based seller already plays it.
> - The Q-learning seller finds that policy in 8 of 10 seeds. In the remaining two it keeps discounting where it should not, so its average is slightly lower: 34.39 ± 1.24 versus 34.97 ± 0.25 per step.
> - The causes are measurable: a constant learning rate keeps Q-values noisy, and the observation hides the rival's delayed reaction.
> - When the market changes (an aggressive rival, other opponent types) the fixed rules fall up to 4.5 per step below the best response, while the learners stay within roughly 0.1–1 of it. A learner trained against a pool of rivals with a shrinking step size was the most robust seller tested.

2 The Market and the Two Sellers

2.1 Structure of the simulation

Figure 1 shows one time step. Both sellers see their own observation, choose a move *simultaneously* without seeing the other's choice, and the market answers with a profit and a new observation for each.

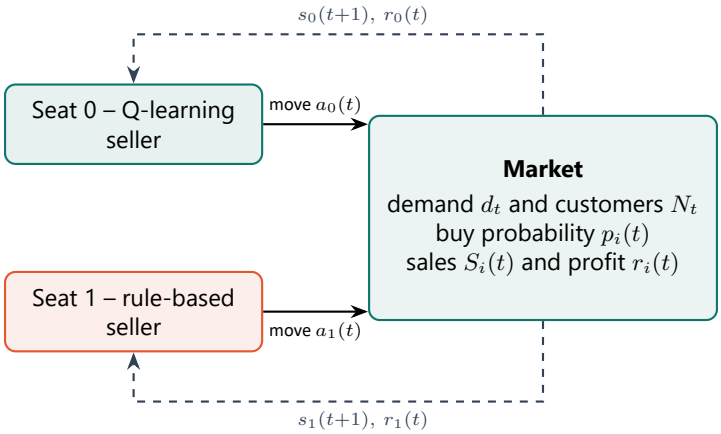


Figure 1. One step of the two-seller market. Solid arrows: simultaneous moves. Dashed arrows: profit and the next observation returned by the market.

Observation (state).
Seller i sees $s_i(t) = (d_t, c_i(t))$: the demand level $d_t \in \{0, 1\}$ (quiet, busy) and the rival's *previous* move $c_i(t) \in \{0, 1\}$. This gives four states per seller.

Move (action).
$a_i(t) \in \{0, 1\}$: keep the price or discount.

Buying probability.
Each of the N_t potential customers buys with probability $p_i(t) = \text{clip}\big(0.30 + 0.25\, d_t + 0.15\, a_i(t) - 0.10\, a_{-i}(t),\ 0.05,\ 0.95\big)$, where $a_{-i}(t)$ is the rival's *current* move.

Reward.
$N_t = 10$ in quiet and 20 in busy periods; sales $S_i(t) \sim \text{Binomial}(N_t, p_i(t))$; price $P = 10$, unit cost $C = 5$, discount $\delta = 0.2$. Profit $r_i(t) = S_i(t)\,\big(P(1 - \delta a_i(t)) - C\big)$ – 5 per sale at full price, 3 per discounted sale.

Transition.
The next demand level is a fair coin flip, independent of the moves; each seller's current move becomes the rival's previous move. An episode lasts 20 steps and starts with both previous moves equal to 0.

2.2 What the model already tells us

Multiplying customers, buying probability and margin gives the expected profit of one step (Table 1); the function `expected_profit` computes it.

Table 1. Expected one-step profit $E[r] = N_t\, p_i$ (margin) for each demand level and each current rival move.

Demand d	Rival's current action	$E[r \mid a = 0]$	$E[r \mid a = 1]$	Better action
low	no discount	15.00	13.50	no discount
low	discount	10.00	10.50	discount
high	no discount	55.00	42.00	no discount
high	discount	45.00	36.00	no discount

Keeping the price is better in three of the four cases. The single exception (quiet demand while the rival discounts, a gain of only $+0.5$) never occurs against the reference rule, which does not discount in quiet periods. A discount also carries a *delayed* cost: if demand is busy at the next step the rule copies it, and our own profit falls from 55 to 45. Against the rule-based rival the optimal policy is therefore **never discount**, worth $0.5 \times 15 + 0.5 \times 55 = 35$ per step. This figure is the yardstick for the rest of the report; if neither seller ever discounts, both earn 35.

2.3 Seller A: tabular Q-learning

The learner stores one value $Q(s, a)$ for each of the $4 \times 2 = 8$ state–move pairs, all initialised to zero. It plays ε-greedily (a random move with probability ε, otherwise the larger Q, ties broken at random) and after each step applies

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)\Big],$$

with the bootstrap term dropped on the last step of an episode. Reference values: $\alpha = 0.10$, $\gamma = 0.90$, $\varepsilon_{\text{ep}} = \max(0.02,\ 0.20 \cdot 0.995^{\text{ep}})$.

```
def update(self, obs, action, reward, next_obs, done=False):
    d, c = obs
    nd, nc = next_obs
    current = self.q[d, c, action]
    bootstrap = 0.0 if done else self.gamma * np.max(self.q[nd, nc])
    target = reward + bootstrap

    self.visits[d, c, action] += 1
    rate = 1.0 / self.visits[d, c, action] if self.step_rule == "visit" else self.lr
    self.q[d, c, action] = current + rate * (target - current)
```

The `"visit"` branch ($\alpha = 1/N$) is only used by the improved learner of Question 10; with the default `"constant"` rule the update is the reference one.

2.4 Seller B: rule-based

$$\pi^{\text{rule}}(d, c) = \begin{cases} \text{keep}, & d = 0 \quad \text{(quiet demand)}, \\ \text{discount}, & d = 1 \text{ and } c = 1 \quad \text{(busy, rival discounted: match it)}, \\ \text{keep}, & d = 1 \text{ and } c = 0 \quad \text{(busy, rival did not discount)}. \end{cases}$$

```
class RuleSeller(FixedSeller):
    def choose(self, obs, explore=True):
        demand, rival_last = obs
        if demand == 0:
            return KEEP
        if rival_last == 1:
            return CUT
        return KEEP
    # update() is inherited from FixedSeller and does nothing: no learning
```

3 Method

3.1 Program layout

All code is in one file, `code/agent02_experiment.py` (NumPy and Matplotlib only). It defines the market (`PriceWarMarket`), the learners (`TabularQSeller`, `MemoryQSeller`), the non-learning sellers (`RuleSeller`, `AggressiveSeller`, `MirrorSeller`, `ConstantSeller`, `CoinFlipSeller`), one `train` and one `evaluate` function, and one `run_*` function per experiment. Every seller exposes `choose(obs, explore)` and `update(…)`, so one loop serves RL-vs-rule, rule-vs-rule and RL-vs-RL. Running the script rebuilds all CSV files, figures and tables in about 25 minutes; the tables in this report are pulled in with `\input` from files the script writes, so no result was typed by hand.

3.2 Settings

Table 2. Settings. Reference values are kept unless an experiment changes them explicitly.

Setting	Value
Sellers	2 (seat 0: seller under test, seat 1: rival)
Training length	5,000 episodes $\times$ 20 steps = 100,000 steps
Learning rate α / discount factor γ	0.10 / 0.90
Exploration	$\varepsilon = \max(0.02,\ 0.20 \cdot 0.995^{\text{episode}})$
Price / unit cost / discount	10 / 5 / 20%
Baseline seed (experiments A, B)	42 (market), $42 + 10{,}000$ (learner)
Repeated runs (experiments C–E, Q8–Q10)	seeds 11, 22, ..., 110 (10 seeds)
Evaluation	500 episodes (10,000 steps), $\varepsilon = 0$, learning off
Evaluation seed	training seed $+ 800{,}000$ (baseline: $42 + 500{,}000$)
Learning-curve window	$K = 100$ episodes

3.3 Evaluation protocol

Reward collected during training mixes the quality of the policy with the cost of exploring. I therefore evaluate in a separate run: the Q-table is frozen, $\varepsilon = 0$, and the market is rebuilt from its own seed.

Paired comparison. In the reference experiment the learner (seat 0) and the rule (seat 1) each face a different opponent behaviour, so their rewards are not directly comparable. To compare the two *policies* I put both in seat 0 against the same rule-based rival and the same evaluation seed ("RL vs Rule" and "Rule vs Rule"). The market draws one demand value and two binomial samples per step no matter which moves are played, so both runs see the identical demand sequence and random stream (common random numbers). Any difference in reward is then caused by the policies alone.

```
def evaluate(cfg, seat0, seat1, seed, episodes=None):
    market = PriceWarMarket(cfg, seed=seed)          # own evaluation seed
    ...
    for ep in range(episodes):
        obs = market.reset()
        for _ in range(horizon):
            moves = [seats[i].choose(obs[i], explore=False) for i in range(2)]
            obs, rewards, _ = market.step(moves)     # no update(): Q stays frozen
            ...
```

Metrics. Mean reward per step $\overline{R}_i = \frac{1}{T}\sum_{t=1}^{T} r_i(t)$; total reward; counts of discount and keep moves; the learned Q-table and its greedy policy; for learning curves, the mean reward per step over blocks of $K = 100$ episodes. Across seeds I report the mean and sample standard deviation of $\overline{R}_i$ and the number of seeds whose final greedy policy never discounts.

4 Findings

4.1 Experiment A – one baseline run (seed 42)

Table 3. Greedy evaluation of the baseline sellers: 500 episodes, 10,000 steps, evaluation seed $42 + 500{,}000$.

Match (seat 0 vs seat 1)	Agent	Total reward	Mean reward/step	Discounts	No discounts	Discount rate
RL vs Rule	RL (seat 0)	351,275	35.127	0	10000	0.000
RL vs Rule	Rule-based (seat 1)	349,685	34.968	0	10000	0.000
Rule vs Rule	Rule-based (seat 0)	351,275	35.127	0	10000	0.000
Rule vs Rule	Rule-based (seat 1)	349,685	34.968	0	10000	0.000

Table 4. Q-table of the baseline learner after 5,000 training episodes (seed 42); visits are counted over the whole run.

State (d, c)	$Q(s, 0)$	$Q(s, 1)$	Visits $(a{=}0, a{=}1)$	Greedy action
(0, 0)	228.30	178.85	(33,659, 12,490)	no discount
(0, 1)	177.99	207.21	(2,501, 1,522)	discount
(1, 0)	246.88	235.23	(42,848, 3,103)	no discount
(1, 1)	211.42	242.96	(3,468, 409)	discount

- In evaluation the learner never discounts, and its total reward (351,275) matches the rule-based seller in the same seat to the last unit: both policies played identical moves on all 10,000 steps. This confirms that the pairing works and that, for this seed, the learner reached the benchmark policy.
- The greedy policy (Table 4) keeps the price in the two states with $c = 0$ and discounts in the two with $c = 1$. The latter resembles the rule's "match the rival" behaviour, but those states receive only about 4% of the training visits (2,501 + 1,522 and 3,468 + 409 out of 100,000) and are *never* reached in greedy play, because the rival only discounts after we do. The choice there is weakly learned and irrelevant for the score.
- During training (with exploration) the totals were 3,345,548 for the learner and 3,283,930 for the rule, i.e. 33.46 and 32.84 per step. The learner discounted on 17.5% of training steps, the rule on 8.3% – and the rule did so only in reaction to the learner's exploratory discounts, a small illustration of how one seller's behaviour shapes the other's outcome.

4.2 Experiment B – the learning curve

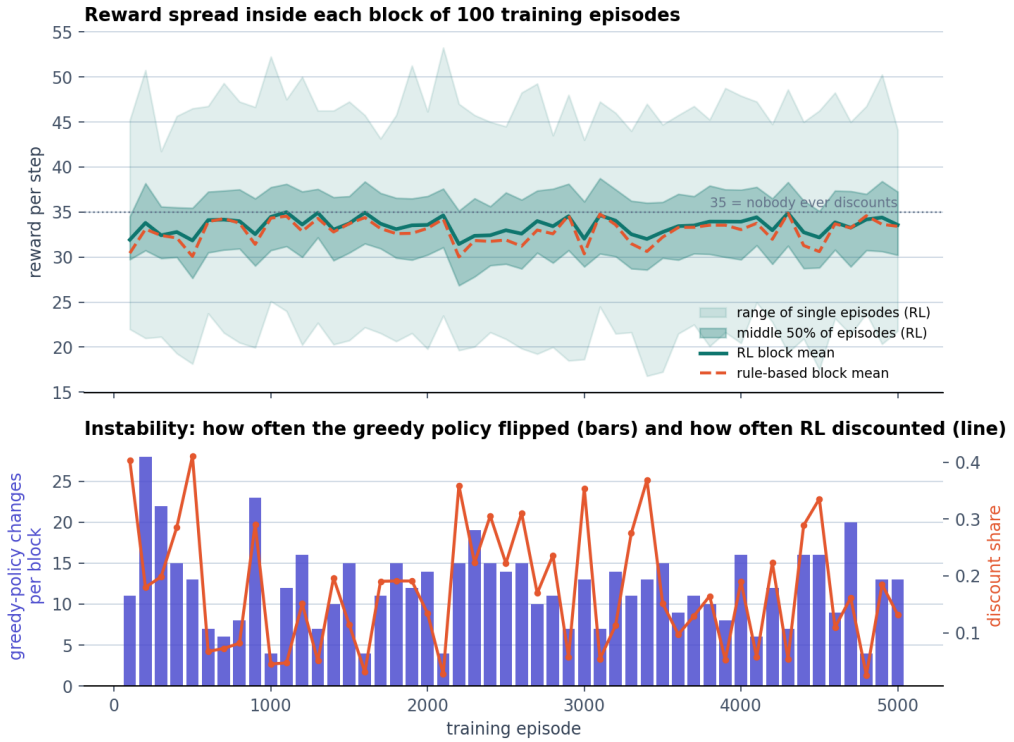


Figure 2. Baseline training, seed 42. Top: for every block of $K = 100$ episodes, the range of single-episode rewards (pale band), the middle 50% (darker band) and the block mean, next to the rule-based seller's block mean. Bottom: number of greedy-policy changes inside each block (bars) and share of steps on which the learner discounted (line).

The block-averaged reward rises from about 32 in the first block to 33–35 after roughly 500 episodes but then *keeps wandering* between about 31 and 35 until the end (Figure 2). Single episodes range from about 20 to 50 because demand is random at every step. The lower panel explains the dips: exploration reaches its floor $\varepsilon = 0.02$ after about 460 episodes, so random exploration causes a discount on only 1% of steps, yet the discount share keeps jumping between 3% and 37%. The bars in the lower panel show that these discounts come from the *greedy* policy itself, which switched 606 times during training (a switch is counted whenever the greedy move of any state differs between two consecutive episodes). Each time the greedy move in state $(0, 0)$ or $(1, 0)$ flips to "discount", the rule retaliates and both sellers lose, which is why the two reward curves move together.

A good-looking final block is therefore no proof of convergence: the policy tested in Experiment A is a snapshot of a table that is still changing. It happened to be a good snapshot for seed 42; Experiment E shows that this is not true for every seed.

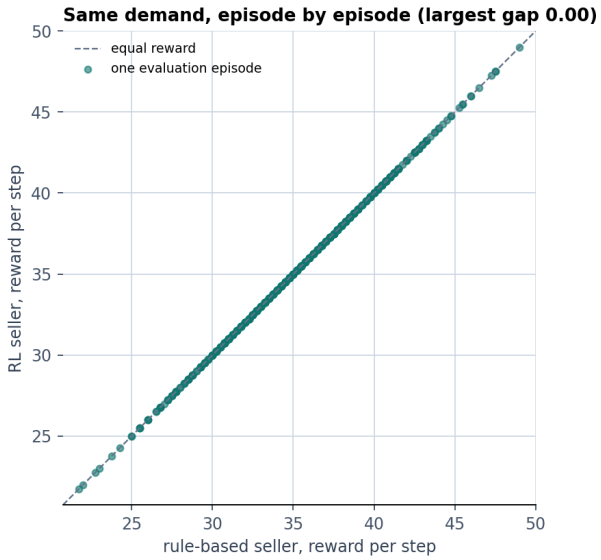


Figure 3. Paired scatter of the seed-42 evaluation: each point is one of the 500 episodes, with the rule-based seller's reward on the horizontal axis and the learner's on the vertical axis. All points lie on the diagonal because both policies played identical moves on the same demand sequence.

4.3 Experiment C – learning rate

Table 5. Learning rate, 10 seeds per row. Train reward: mean over all 5,000 training episodes. "Never discount": seeds whose final greedy policy never discounts in any state. Policy changes: episodes after which the greedy policy differed from the previous one.

α	Train reward	Eval. reward	Eval. std	Discount rate	"Never discount"	Policy changes
0.01	32.441	33.634	3.193	0.150	1/10	21.5
0.1	33.608	34.390	1.239	0.091	4/10	584.1
0.5	31.812	32.045	3.509	0.287	3/10	1769.9

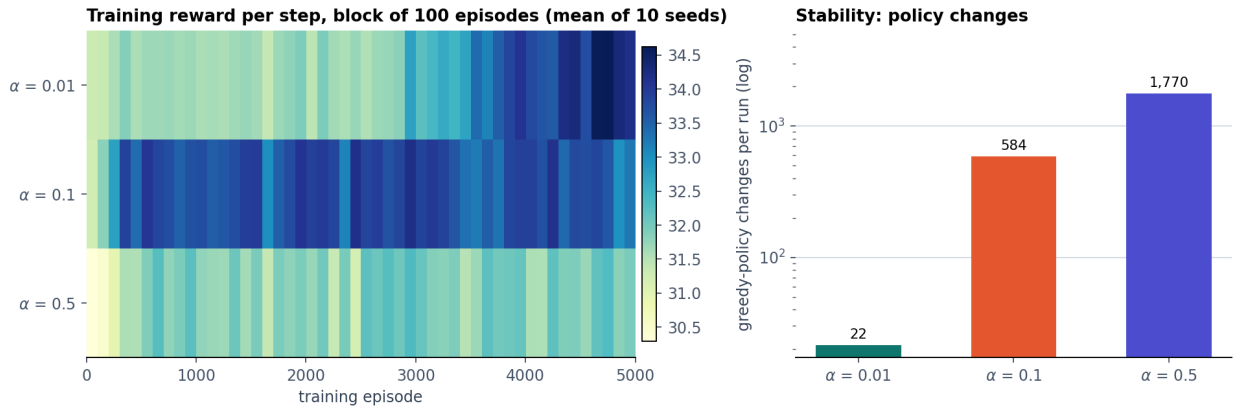


Figure 4. Left: heatmap of the block-averaged training reward (mean over 10 seeds), one row per learning rate; darker means more profit. Right: mean number of greedy-policy changes per run, on a logarithmic scale.

$\alpha = 0.10$ gives the best evaluation reward and by far the smallest spread. The heatmap in Figure 4 shows the timing: $\alpha = 0.10$ reaches high training reward within about 500 episodes, $\alpha = 0.01$ only in the last 1,500 episodes (it is still climbing when training stops), and $\alpha = 0.50$ never does. With $\alpha = 0.01$ the Q-values spend much of the run climbing from 0 towards values around 150–260. While they climb, the move chosen more often has simply received

more updates and looks better, so the greedy choice reinforces itself: for seed 11 the learner played "discount" 31,019 times and "keep" only 4,358 times in state $(0, 0)$ before "keep" finally overtook it. Its policy changed just 21.5 times per run – stable, but frequently stable in the wrong place. With $\alpha = 0.50$ every new sample moves Q strongly: about 1,770 policy changes per run (one every three episodes), the lowest training reward, and a final policy that is close to a coin flip wherever the two Q-values are near each other. Questions 6 and 7 return to this.

4.4 Experiment D – exploration

Table 6. Exploration settings, 10 seeds each, $\alpha = 0.10$. Rewards are means over seeds; std is across seeds.

Exploration	Train reward	Train std	Eval. reward	Eval. std	"Never discount"
Reference decay 0.20→0.02	33.608	0.144	34.390	1.239	4/10
Fixed ε = 0.05	32.909	0.205	34.122	1.346	4/10
Fixed ε = 0.30	30.888	0.101	29.654	3.264	0/10
Greedy, ε = 0	31.094	3.069	31.029	3.099	3/10

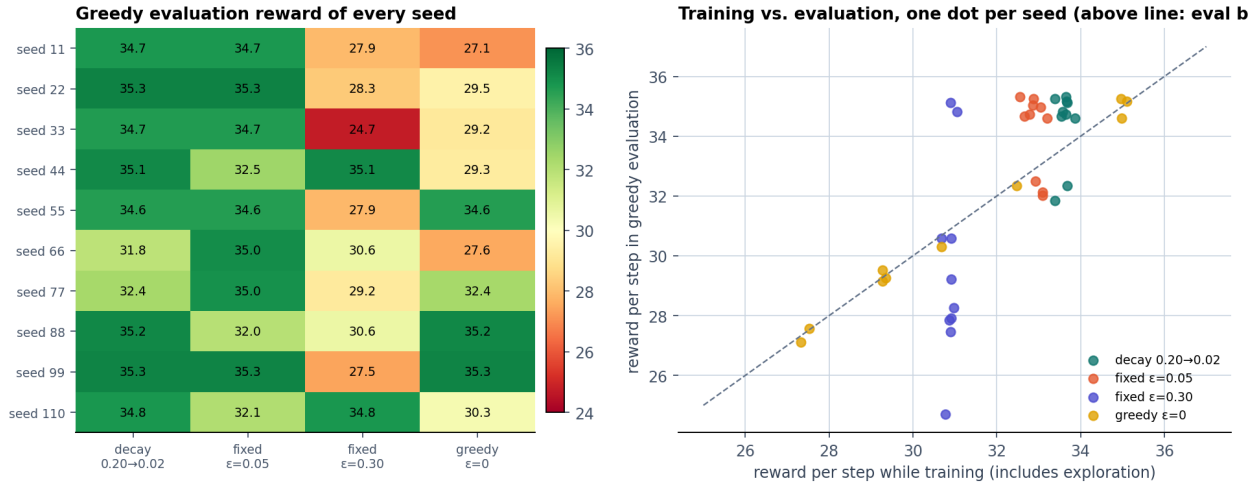


Figure 5. Left: greedy evaluation reward of every seed under each exploration schedule (green = high, red = low). Right: reward while training against reward in greedy evaluation, one dot per seed; dots above the dashed diagonal did better in evaluation than in training.

The decaying reference schedule is best in training and in evaluation. A fixed $\varepsilon = 0.05$ is close in evaluation (34.12) but loses about 0.7 per step in training to its random moves. A fixed $\varepsilon = 0.30$ has the lowest training reward (30.89) and, perhaps surprisingly, an even lower evaluation reward (29.65): none of its 10 final policies is "never discount". The purely greedy learner ($\varepsilon = 0$) never changed its policy at all: all Q-values start at 0 and every profit is positive, so the first move tried in each state becomes the greedy move for ever. Its result depends entirely on the seed (std 3.07). Question 4 explains these effects.

4.5 Experiment E – paired comparison over ten seeds

Table 7. Summary table required by Section 7.6 of the assignment. Every seller sits in seat 0 against the reference rule-based rival; greedy evaluation of 500 episodes per seed; mean and std over 10 seeds. Discount rate: share of evaluation steps with a discount.

Agent / configuration	Mean reward	Std. dev.	Discount rate	"Never discount" policy
RL, baseline ($\alpha = 0.10$, decaying ε)	34.390	1.239	0.091	4/10
RL, low learning rate ($\alpha = 0.01$)	33.634	3.193	0.150	1/10
RL, high learning rate ($\alpha = 0.50$)	32.045	3.509	0.287	3/10
RL, fixed $\varepsilon = 0.05$	34.122	1.346	0.132	4/10
RL, fixed $\varepsilon = 0.30$	29.654	3.264	0.495	0/10
RL, greedy $\varepsilon = 0$	31.029	3.099	0.436	3/10
Rule-based	34.970	0.253	0.000	–

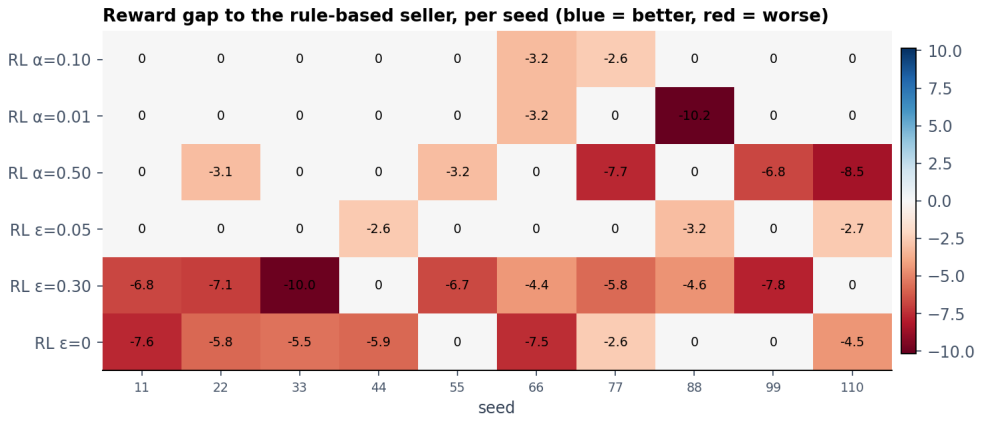


Figure 6. Reward gap to the rule-based seller for every configuration and seed (same demand sequence). Blue cells: better than the rule, red: worse, 0: identical.

The rule-based seller earns 34.97 ± 0.25 per step, in line with the expected 35; its spread comes only from random demand and sales because its policy is fixed. The best learner (the reference configuration) earns 34.39 ± 1.24. Since the comparison is paired I also computed the per-seed difference learner $-$ rule: exactly 0 for 8 seeds and -3.19 and -2.61 for seeds 66 and 77 (mean -0.58, std 1.23). The learner thus matches the rule or does worse, never better – as expected, because the rule already plays the best response. Only 4 of the 10 learned policies are "never discount" in *all* four states; the other 4 that earn the same reward discount only in the unreachable states with $c = 1$.

5 Answers to the Ten Laboratory Questions

Q1 Modelling the agents and the environment

- **Agents:** two sellers. Seat 0 is the seller under test (Q-learning); seat 1 is the rival (rule-based in the reference set-up).
- **Environment:** the market – customers, random demand, buying-probability and profit rules, and the record of last step's moves (`PriceWarMarket`).
- **State space:** $\mathcal{S} = \{0,1\} \times \{0,1\}$ (demand, rival's previous move): $(0,0), (0,1), (1,0), (1,1)$.
- **Action space:** $\mathcal{A} = \{0,1\}$, keep or discount.
- **Reward:** the profit of the step, $r_i = S_i\big(P(1-\delta a_i) - C\big)$ with $S_i \sim \text{Binomial}(N_t, p_i)$.

- **Transition:** d_{t+1} is uniform on $\{0, 1\}$ independent of everything else; $c_i(t+1) = a_{-i}(t)$.

Why this is a multi-agent system. A seller's reward depends on the rival's *current* move (the $-0.10\, a_{-i}$ term) and its next observation contains the rival's move. A single-agent model would have to treat the rival as part of the environment, which is only legitimate while the rival's behaviour does not change. As soon as the rival reacts to us (the rule copies our discounts) or learns (Question 8), the "environment" changes with our own choices. Modelling two agents makes this interaction explicit; in Experiment A the rule discounted on 8.3% of training steps purely in response to exploratory discounts.

Q2 Learning-based versus rule-based decisions

	Q-learning seller	Rule-based seller
How it decides	Plays the move with the higher learned value $Q(s, a)$; sometimes explores.	Looks the move up in three hand-written rules.
Use of experience	Every step updates Q; behaviour changes with data.	Ignores rewards; behaviour never changes.
Adaptability	Relearns a good response when the market changes (Q9: retrained learner 28.76 vs. 27.84 for the rule).	A developer must rewrite it when the market changes.
Cost	100,000 training steps (about 10 s here) and an 8-entry table; far more for large state spaces.	No training; one comparison per decision.
Predictability	Different seeds yield different policies (std 1.24 across seeds).	Identical decisions every time; easy to audit.

The rule-based seller is the better choice when correct behaviour is known and must be guaranteed, explained or audited, or when experimentation is unsafe – for example a legally mandated price floor, "never sell below cost", or a new shop with no sales history. In the reference market the rule happens to be the best response, and learning can only tie it.

The learning seller is the better choice when the best response is unknown or keeps moving and experimenting is cheap – for example a marketplace seller facing many competitors whose pricing changes every season. In the robustness test of Question 10 the learners averaged 1.1–1.7 per step more than the fixed rule across five rival types.

Q3 The Q-learning update

The update moves the current estimate $Q(s, a)$ a fraction α of the way towards a *target*: the reward just received plus the discounted value of the best move in the next state. The bracket $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ is the *temporal-difference (TD) error*: positive if the outcome beat the expectation, negative if it fell short.

For $Q(s, a) = 4$, $\alpha = 0.1$, $r = 6$, $\gamma = 0.9$ and $\max_{a'} Q(s', a') = 5$:

$$\begin{aligned} \text{target} &= 6 + 0.9 \times 5 = 10.5, \\ \text{TD error} &= 10.5 - 4 = 6.5, \\ \text{increment} &= 0.1 \times 6.5 = 0.65, \\ Q_{\text{new}}(s, a) &= 4 + 0.65 = 4.65. \end{aligned}$$

Interpretation. The move did better than believed (10.5 against 4), so its value rises – but only by 10% of the gap, to 4.65. One lucky outcome is not taken at face value; repeated good outcomes keep pushing the value up. With $\alpha = 1$ the value would jump to 10.5 immediately, with $\alpha = 0.01$ it would only reach 4.065.

Q4 Exploration versus exploitation

ε-greedy selection plays a random move with probability ε (exploration) and otherwise the move with the larger Q (exploitation). A Q-value can only improve if the move is actually tried, hence exploration; reward is only earned by using what has been learned, hence exploitation. The experiments (Table 6) show both ways to get it wrong:

- **Too much exploration (fixed $\varepsilon = 0.30$).** Training reward drops to 30.89 (from 33.61) because 15% of steps are random discounts, each of which also triggers the rule's retaliation at the next busy step. Worse, the *learned* policy is poor too (evaluation 29.65, discount rate 0.49, no seed reached "never discount"): random discounts make the rival's next move erratic, the targets become much noisier, and with $\alpha = 0.10$ the small gap between the two moves in quiet periods is swamped. The greedy policy changed 2,429 times per run.
- **Too little exploration ($\varepsilon = 0$).** With zero initial values and positive profits, whichever move is tried first in a state gets a positive value and is chosen for ever (0 policy changes). In 3 of 10 seeds that first choice was "keep" everywhere and the result was good; in the others it locked in a discount somewhere and lost up to 8 per step (27.11 for the worst seed). Mean 31.03, std 3.10 – luck rather than learning.
- **The reference decay** explores widely during the first 500 episodes and then exploits with $\varepsilon = 0.02$, giving the best training (33.61) and evaluation (34.39) reward. A small fixed $\varepsilon = 0.05$ learns almost as well (34.12) but pays for exploration all the time (training 32.91).

Q5 Reward design and business objectives

A discount lowers the price so more visitors buy (buying probability $+0.15$), but each sale earns less. With $P = 10$, $C = 5$, $\delta = 0.20$:

$$\begin{aligned} \text{no discount:} \quad & 10 - 5 = 5 \text{ per sale,} \\ \text{discount:} \quad & 10 \times (1 - 0.20) - 5 = 8 - 5 = 3 \text{ per sale.} \end{aligned}$$

The margin falls by 40%, so a discount only pays if sales grow by a factor of at least $5/3 \approx 1.67$. The gain from the discount is far below that: $0.30 \to 0.45$ in quiet demand is $\times 1.5$, and $0.55 \to 0.70$ in busy demand only $\times 1.27$. That is why "keep" wins in Table 1 except when the rival is already discounting in quiet periods.

Changing the reward changes the behaviour. An agent maximises what the reward measures, not what the business "means". For example:

- *Revenue instead of profit* ($r = S \cdot P_{\text{eff}}$): 8 against 10 per sale with $\times 1.27$–1.5 more sales makes discounting best in every state, so the agent learns to always discount.
- *Profit plus a market-share bonus* ($r = \text{profit} + \lambda S$): for large enough λ the extra sales outweigh the lost margin and the agent discounts more.
- *A smaller discount*, e.g. $\delta = 0.10$ (margin 4): quiet demand $10 \times 0.45 \times 4 = 18 > 15$ and busy demand $20 \times 0.70 \times 4 = 56 > 55$, so a myopic agent would always discount; the final behaviour then depends on whether the rival's retaliation (about $0.5 \times 8 \times \gamma$) outweighs the gain.

Q6 Measuring performance

Mean reward per step, $\overline{R} = \frac{1}{T} \sum_t r(t)$, from the greedy evaluations ($T = 10{,}000$ steps per seed):

Measurement	Q-learning seller	Rule-based seller
Baseline, seed 42, same seat (Table 3)	35.127	35.127
Baseline, seed 42, reference RL-vs-Rule match	35.127 (seat 0)	34.968 (seat 1)
Training phase, seed 42 (with exploration)	33.455	32.839
10 seeds, same seat, mean $\pm$ std (Table 7)	34.390 ± 1.239	34.970 ± 0.253
10 seeds, paired difference learner $-$ rule	-0.580 ± 1.231 (0 in 8 seeds)	

Why a single run is not enough. Seed 42 alone suggests the two sellers are identical (and the reference match even makes the learner look 0.16 better, which is merely the difference between the two seats' random sales). Ten seeds reveal that the learner is worse in 2 of 10 runs, by about 3 per step. Several sources of randomness move a single result: random demand and binomial sales (a 20-step episode ranges from about 20 to 50 per step), the learner's own exploration and tie-breaking, and the moment training stops while the greedy policy is still switching (606 switches

in the seed-42 run). Claiming that one approach is *generally* better additionally requires other markets and rivals (Questions 9 and 10), not only more seeds.

Q7 Learning rate and stability

The learning rate sets how strongly a new observation moves Q: the weight of an observation k updates old is $\alpha(1-\alpha)^k$. With $\alpha = 0.5$ only the last few samples matter; with $\alpha = 0.01$ the estimate averages over hundreds. Over 10 seeds (Table 5, Figure 4):

- $\alpha = 0.01$ **– calm but slow.** Only 21.5 policy changes per run, but the values are still climbing from 0 for much of the run, so the move tried more often looks better simply because it was updated more. Evaluation 33.63 with the largest spread (3.19); only 1 seed learned "never discount". A learner insensitive to noise is insensitive to *useful* information as well.
- $\alpha = 0.10$ **– the best compromise** (34.39 ± 1.24), though still 584 policy changes per run.
- $\alpha = 0.50$ **– fast but unstable.** About 1,770 policy changes per run; training reward 31.81, evaluation 32.05 ± 3.51, discount rate 0.29. In quiet periods the true gap between the moves is a few units while one reward fluctuates by roughly ± 7, so a Q-value that half-resets to each new sample ranks the moves almost at random.

Faster updating therefore did *not* buy stability; it produced excessive fluctuation. With a constant α in a noisy environment the estimates never stop moving. A step size that shrinks with the number of visits (used in Question 10) addresses exactly this.

Q8 Interaction between sellers and non-stationarity

How the rival influences me. The rival's *current* move changes my buying probability by -0.10, costing 5 per step in quiet and 10 in busy periods (Table 1); the rival's *previous* move is part of my observation. Against the rule there is also a feedback loop: my discount now triggers the rival's discount at the next busy step and lowers my next reward. This delayed cost is invisible in my observation, which contains the rival's last move but not my own (see Question 9).

Why independent Q-learning is hard when both learn. Q-learning assumes fixed transition and reward probabilities. If the rival learns too, the reward I receive for the same (s, a) changes as its policy changes: the problem is *non-stationary*. Old samples become outdated, the two learners chase each other, and the usual convergence guarantees no longer apply.

Experiment. I trained two independent Q-learners against each other (same parameters, different random generators) and compared them with a learner facing the fixed rule, over 10 seeds (Figure 7).

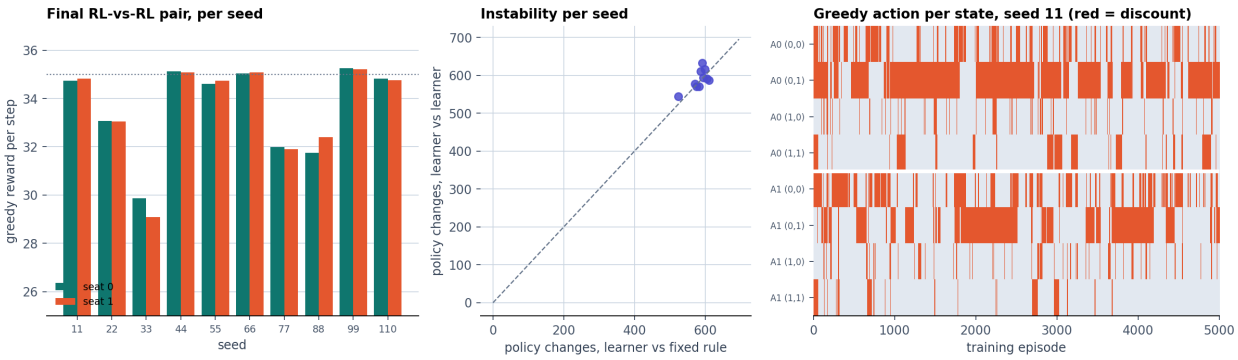


Figure 7. Left: final greedy reward of seat 0 and seat 1 for every seed in the RL-vs-RL pair. Middle: greedy-policy changes per run against a learning rival (vertical) versus a fixed rule (horizontal), one dot per seed. Right: greedy move of each state for both learners (A0, A1) over training, seed 11; red cells mean "discount".

- With two learners the share of discounting steps during training stays higher (about 0.20–0.30 instead of 0.10–0.20, averaged over the 10 seeds): each seller's exploratory discounts make the other's learned values noisier.
- In greedy evaluation of the two final policies against each other (left panel), 6 of 10 pairs settled on "nobody discounts" (about 35 each), while 4 pairs ended with one or both still discounting (as low as 29.1–29.9 for seed

33). Mean rewards were 33.63 ± 1.86 and 33.62 ± 2.01 – lower and more variable than learning against the fixed rule.

- The number of policy changes was almost identical (589 versus 584 per run; in the middle panel the dots scatter around the diagonal). In this tiny problem the noise from a constant α already causes most of the instability, so this counter cannot separate the two effects – an honest limitation of the measurement.

A further experiment to isolate the effect: freeze one learner while the other learns, alternating every M episodes. If performance and stability improve as M grows, non-stationarity is the cause. Another option is to add the rival's recent move frequencies to the observation, making changes in its behaviour visible.

Q9 Limitations of the simulation and a modified market

Limitations of the reference simulation.

1. **Demand is i.i.d.** – a fair coin each step, with no trend, seasonality or persistence, and unaffected by price. Real demand is autocorrelated, so the observation should carry history.
2. **The buying formula is illustrative:** linear, identical for all customers, with fixed effects $+0.15$ and -0.10. Customers cannot switch shops or remember earlier prices, and binomial noise is large compared with the differences between moves.
3. **The observation is incomplete.** The rule reacts to *my* previous move, which is not in my observation, and the step number inside the 20-step episode is missing as well (although the value of the future depends on how many steps remain). From the learner's point of view the problem is not exactly a Markov decision process.
4. **One rival and two moves:** the learner is trained and tested against the same rule, so the results say nothing about other rivals; prices are binary (0% or 20%).
5. **Short training and a single objective:** profit per step only – no inventory, cost of changing prices or long-term customer value.

Implemented modification: an aggressive rival. I replaced the rival's rule with "discount whenever demand is busy", a plausible market change. Against this rival, never discounting is still the best response, with expected profit $0.5 \times 15 + 0.5 \times 45 = 30$. The reference rule-based seller, however, sees "busy and rival discounted last step" on 25% of steps, matches the discount, and earns 36 instead of 45 on those steps: expected value $30 - 0.25 \times 9 = 27.75$.

Table 8. Modified market: aggressive rival in seat 1. Greedy evaluation, 10 seeds.

Agent in seat 0 (opponent: aggressive competitor)	Mean reward	Std. dev.	Discount rate
RL retrained against the aggressive competitor	28.759	1.189	0.322
RL trained on the reference market (not retrained)	29.315	0.922	–
Rule-based	27.842	0.155	0.237

Effect. The measured reward of the rule (27.84) agrees with the calculation (27.75): its "match the rival" reflex, harmless in the reference market, now costs about 2.2 per step. On average both learners beat it – the one retrained in the new market by $+0.92 \pm 1.20$ per step (paired over seeds: better in 6 of 10, equal in 1, slightly worse in 3) and the one trained in the old market and moved over unchanged by $+1.47$. Neither reliably reached the optimum of 30: the retrained learner still discounted on 32% of steps, for the same reason as in Experiment E (noisy values under a constant α). The two learner variants differ by less than one standard deviation, so the data do not show that retraining helped here. What is clear is the change of ranking: a rule that suits one market can be the weakest choice in another, while a learner adapts without being rewritten.

Q10 Designing a better agent

Proposal. The experiments expose three weaknesses of the reference learner: noisy values caused by a constant α, exposure to only one rival, and an observation that hides its own previous move. The proposed agent targets each:

- **State:** (d_t, c_t, a_{t-1}) – demand, the rival's last move and the seller's own last move, so a rival's delayed reaction becomes observable (16 Q-values). Possible extensions: a demand trend, the rival's discount frequency over the last k steps, the steps remaining in the episode.
- **Actions:** discount or keep (could be extended to several discount levels).
- **Reward:** profit per step, matching the business goal (Question 5 explains why not revenue).
- **Learning:** Q-learning with step size $\alpha = 1/N(s,a)$ that shrinks with the number of visits, so values average out the sales noise instead of fluctuating, and training against a **pool of rivals** (rule, aggressive, tit-for-tat, always-discount, random), one drawn at random per episode.
- **Evaluation:** greedy and frozen; against every rival type separately; 10 seeds; report mean, std, the *worst case* over rivals and the gap to the best response computed from the model.

Robustness test. I implemented the proposal in two stages – rival pool with $\alpha = 1/N$, then additionally the extended state – and evaluated all sellers against five rivals (Table 9, Figure 8).

Table 9. Robustness test: mean reward per step $\pm$ std over 10 seeds, greedy evaluation against each rival. Last row: best expected reward against that rival, computed from the model.

Agent	Rule	Aggressive	Tit-for-tat	Always disc.	Random	Average	Worst
Rule-based	34.98 ± 0.27	27.89 ± 0.14	34.98 ± 0.27	23.23 ± 0.16	28.61 ± 0.21	29.94	23.23
RL baseline (trained vs rule)	34.42 ± 1.25	29.32 ± 0.85	34.33 ± 1.46	26.70 ± 1.74	30.61 ± 1.06	31.08	26.70
RL + opponent pool + $\alpha = 1/N$	34.72 ± 0.77	29.83 ± 0.24	34.70 ± 0.82	27.53 ± 0.22	31.17 ± 0.22	31.59	27.53
Improved RL (+ own previous action in state)	34.02 ± 1.27	29.61 ± 0.44	33.80 ± 1.55	27.56 ± 0.22	31.04 ± 0.35	31.21	27.56
Best response (from the model)	35.00	30.00	35.00	27.75	31.25	31.80	27.75

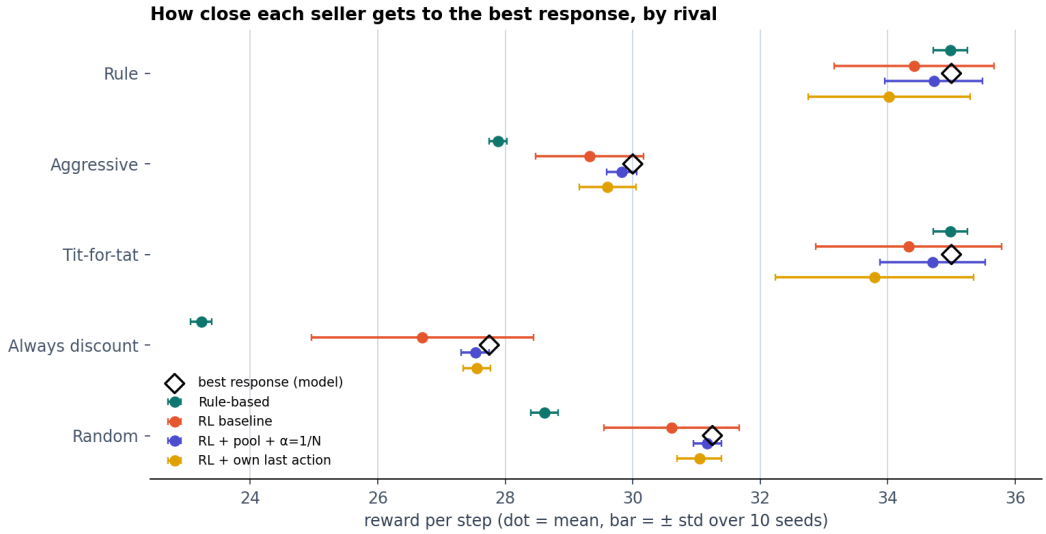


Figure 8. Mean reward per step of every seller against every rival (dot) with $\pm$ 1 std over 10 seeds (bar); the open diamonds mark the best response computed from the model.

- The **rule-based seller** is optimal against the rule and tit-for-tat but loses 4.5 per step against an always-discounting rival (23.23 vs. 27.75), because it then matches the rival's discount even in busy periods.
- The **reference learner**, trained only against the rule, already transfers better (average 31.08, worst case 26.70) but is noisy (std up to 1.74).
- **Rival pool +** $\alpha = 1/N$ is the most robust: average 31.59 and worst case 27.53, within 0.1–0.3 of the best response against every rival (31.80 and 27.75), with std of only 0.22–0.24 against stateless rivals. Against the rule it earns 34.72, closer to 35 than the reference learner (34.42).
- **Adding the own previous move did not help** (average 31.21; 34.02 against the rule). The hypothesis that the hidden delayed reaction is the main problem is *not* supported by this measurement. A plausible but untested

explanation: the 16-entry table spreads the same data over twice as many cells, and with $\alpha = 1/N$ early bootstrapped targets keep a long-lasting weight. Testing it would require longer training or a gentler $\alpha = 1/N^{0.6}$.

One limitation of this test: all five evaluation rivals were also in the training pool, so it measures robustness against *known* rival types. A stricter test would hold one type out of training and evaluate against it.

My recommendation is therefore the 8-state Q-learner trained against a rival pool with a shrinking step size. It beats the reference learner against every rival, beats the rule on average and in the worst case, and gives up only 0.26 per step to the rule in the single market the rule was designed for.

6 Reflection

Consistency versus fit. The rule-based seller was the most consistent in every test (std 0.14–0.27, due only to random demand). Its quality, however, depended wholly on whether its rules matched the rival: the best possible against the reference rule, and 4.5 per step below the best response against an always-discounting rival.

Learning does not guarantee optimality. The learner found the optimal policy in 8 of 10 seeds and never beat the rule, which was already optimal. A constant learning rate, a short horizon of very noisy rewards and an incomplete observation kept the greedy policy changing until training ended.

Randomness. The same fixed policy earns 34.97 ± 0.25 across ten seeds, and single episodes range from 20 to 50 per step. Paired evaluation with common random numbers removed this noise from the learner-vs-rule comparison: on seed 42 the two totals were identical to the last unit.

Interacting learners. Learners shape each other's data: the rule discounted on 8.3% of training steps only because of the learner's exploration, and with two learners 4 of 10 pairs ended with a worse joint outcome.

Training versus evaluation. The two measure different things. With $\varepsilon = 0.30$ training reward was 30.89 and evaluation 29.65; with the reference schedule 33.61 and 34.39. A training curve shows what the agent earned *while* learning, including the price of exploring; only a frozen, separate evaluation shows the quality of the learned policy.

Observation versus explanation. The numbers in this report are observations. Explanations such as "noise from a constant α" or "the hidden delayed reaction" rest on indirect evidence (policy-change counts, visit counts) and are not proven. One of them – that the hidden own move is the main problem – was tested in Question 10 and not confirmed.

7 Conclusion

I implemented a two-seller market, a tabular Q-learning seller and a rule-based seller, and compared them through a separate, paired, multi-seed evaluation. In the reference market the hand-written rule is already the best response, and Q-learning reaches the same policy in most runs (34.39 ± 1.24 vs. 34.97 ± 0.25 per step). Learning rate and exploration schedule matter: $\alpha = 0.10$ with decaying ε worked best, while too much exploration, none, or learning that is too slow or too fast produced worse and less reliable policies. When the rival changes, the advantage reverses: fixed rules lose up to 4.5 per step, whereas learners adapt. A learner trained against a pool of rivals with a decreasing step size came within 0.3 per step of the best response against every rival tested. The broader lesson is that which agent wins is a statement about the environment and its assumptions as much as about the algorithm.

Appendix: Reproducing the Results

```
pip install numpy matplotlib
cd agent_02/code
python agent02_experiment.py                    # all experiments (~25 min), writes ../results/
python agent02_experiment.py --only robustness  # just the Question 10 study
cd ../report && xelatex agent02.tex             # run twice to refresh the contents
```

The notebook `notebook/agent02_demo.ipynb` runs the same experiments step by step with commentary. Seeds

and the evaluation protocol are given in Section 3; all per-seed and summary CSV files are in `agent_02/results/`.

References

[1] Intelligent System Development, *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*, assignment text and reference implementation, 2026.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018 (Q-learning, ε-greedy selection, step-size parameters).